

TPI POO - De Java a Python

PARTE 1 - Diagnóstico de java-ismos

Java-ismo	Dónde	Inversión	Síntoma observable
Getters/setters preventivos	Lado	@property	Obliga usar métodos para acceder/mutar.
Olvido de <code>super()</code>	Poligono	Compilador vs Runtime	<code>_construida</code> no se inicializa al nacer.
Argumentos mutables	Poligono	Declaración vs Runtime	Instancias comparten la misma lista en memoria.
Sobrecarga simulada	Triangulo/Cuadrado	Duck Typing / Opcionales	Uso de <code>if/instance</code> en vez de args por defecto.
Type hint falso	Poligono.area	Compilador vs Runtime	Devuelve <code>str</code> prometiendo <code>int</code> sin fallar.
Acumulador manual	Poligono.perimetro	Expresiones idiomáticas	Menos eficiente que delegar la suma a <code>sum()</code> .
Rompe encapsulamiento	Poligono.getLados	Copia Defensiva	Devuelve referencia directa a la lista interna.
Falso "static" mutable	Poligono.catalogo	Declaración vs Runtime	Estado global acoplado; debe ir en <code>Taller</code> .

Demostración @property: Por el Principio de Acceso Uniforme, pasar de un atributo público simple (`self.longitud = x`) a un encapsulamiento con `@property` y `@longitud.setter` para validar valores no altera al cliente, que sigue operando con la misma sintaxis: `mi_lado.longitud = 10`.

PARTE 2 - Relaciones estructurales Aunque la sintaxis de asignación sea idéntica (`self._algo = algo`), la diferencia estructural se delata en el ciclo de vida y la instanciación:

- **Composición (Polígono - Lado):** El contenedor crea y controla el ciclo de vida de sus partes al nacer.
 - Línea: `c = Cuadrado("Cuadrado", "azul", [Lado(2), Lado(2), Lado(2), Lado(2)])`
- **Agregación (Taller - Polígono):** El contenedor recibe referencias de objetos construidos externamente de forma independiente.
 - Línea: `mi_taller.recibir(c)`
- **Asociación (Lado - Etiqueta):** Dependencia opcional (multiplicidad 0..1) resuelta por parámetro por defecto.
 - Línea: `def __init__(self, longitud, etiqueta=None):` (en clase `Lado`)

PARTE 3 - Herencia por dominio (PoligonoRegular): Se eliminó por redundancia semántica (un regular de 4 lados ES un cuadrado) y por ser un artefacto del tipado estricto de Java. Se reemplazó por la función de fábrica `fabricar_poligono_regular()`, aprovechando las listas heterogéneas de Python para instanciar la subclase adecuada sin jerarquías artificiales.

PARTE 4 - ABC vs. Protocol

- **abc.ABC (Contrato Nominal):** Fuerza herencia explícita y "falla temprana" en subclases propias (*Triangulo*, etc.).
- **typing.Protocol (Contrato Estructural):** Formaliza el *Duck Typing* para integrar librerías de terceros cerradas (*PlanoCAD*) sin modificar código ajeno.
- **¿Lenguaje o Dominio?:** Lo decide el dominio. Entidades conceptualmente distintas no deben forzar una jerarquía de herencia artificial solo para complacer a un compilador.

Tabla de Equivalencias (Java vs Python)

Elemento en Java	Código Python	Tipo	Por qué / Criterio de Diseño
<code>public class PoligonoRegular extends Poligono</code>	<code>def fabricar_poligono_regular(...)</code>	Rediseño	Reemplaza jerarquía forzada por una Factory Function, aprovechando listas heterogéneas.
Getters/Setters preventivos (<code>getLongitud</code>)	<code>@property</code> y <code>@longitud.setter</code>	Rediseño	Aplica el Principio de Acceso Uniforme: valida sin alterar el contrato del cliente.
Interfaz <code>Exportable</code> (Herencia explícita)	<code>class Exportable(Protocol):</code>	Rediseño	Contrato estructural (Duck Typing) para integrar <i>PlanoCAD</i> sin modificar código ajeno.
Bucle acumulador imperativo	<code>sum(l.longitud for l in ...)</code>	Traducción	Reemplaza lógica manual por expresiones generadoras idiomáticas nativas.
Atributo de clase <code>static (catalogo)</code>	Lista en clase <code>Taller (self._poligonos)</code>	Rediseño	Elimina estado global acoplado, delegando la responsabilidad de inventario al gestor.
Llamada implícita al padre	<code>super().__init__(nombre, color)</code>	Traducción	Exige invocación explícita al no existir inicialización mágica de constructores.